

```
In [2]: # =====
# PRODUCT OPTIMIZATION & REVENUE ANALYSIS
# Unified Mentor Internship
# =====

import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import plotly.express as px

# Display all columns
pd.set_option("display.max_columns", None)
pd.set_option("display.max_rows", 20)

print("Libraries Imported Successfully ✅")
```

Libraries Imported Successfully ✅

```
In [17]: # Load Dataset

file_path = "../downloads/Afficionado Coffee Roasters.xlsx"

df = pd.read_excel(r"C:\Users\Manasa Prakash\Downloads\Afficionado Coffee Roaste
print("Dataset Loaded Successfully ✅")
```

Dataset Loaded Successfully ✅

```
In [18]: df.head()
```

```
Out[18]:
```

	transaction_id	year	transaction_time	transaction_qty	store_id	store_location	prod
--	----------------	------	------------------	-----------------	----------	----------------	------

0	1	2025	07:06:11	2	5	Lower Manhattan	
---	---	------	----------	---	---	-----------------	--

1	2	2025	07:08:56	2	5	Lower Manhattan	
---	---	------	----------	---	---	-----------------	--

2	3	2025	07:14:04	2	5	Lower Manhattan	
---	---	------	----------	---	---	-----------------	--

3	4	2025	07:20:24	1	5	Lower Manhattan	
---	---	------	----------	---	---	-----------------	--

4	5	2025	07:22:41	2	5	Lower Manhattan	
---	---	------	----------	---	---	-----------------	--



```
In [19]: print("Rows :", df.shape[0])
print("Columns :", df.shape[1])
```

Rows : 149116

Columns : 11

```
In [20]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 149116 entries, 0 to 149115
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype  
---  -
0   transaction_id         149116 non-null  int64  
1   year                   149116 non-null  int64  
2   transaction_time       149116 non-null  object  
3   transaction_qty        149116 non-null  int64  
4   store_id               149116 non-null  int64  
5   store_location         149116 non-null  object  
6   product_id             149116 non-null  int64  
7   unit_price             149116 non-null  float64 
8   product_category       149116 non-null  object  
9   product_type           149116 non-null  object  
10  product_detail         149116 non-null  object  
dtypes: float64(1), int64(5), object(5)
memory usage: 12.5+ MB
```

```
In [21]: df.isnull().sum
```

```

Out[21]: <bound method DataFrame.sum of
transaction_qty store_id \
0 False False False False False
1 False False False False False
2 False False False False False
3 False False False False False
4 False False False False False
... ... ... ... ...
149111 False False False False False
149112 False False False False False
149113 False False False False False
149114 False False False False False
149115 False False False False False

store_location product_id unit_price product_category \
0 False False False False
1 False False False False
2 False False False False
3 False False False False
4 False False False False
... ... ... ... ...
149111 False False False False
149112 False False False False
149113 False False False False
149114 False False False False
149115 False False False False

product_type product_detail
0 False False
1 False False
2 False False
3 False False
4 False False
... ... ...
149111 False False
149112 False False
149113 False False
149114 False False
149115 False False

[149116 rows x 11 columns]>

```

```

In [22]: print("Duplicate Records :", df.duplicated().sum())

```

```

Duplicate Records : 0

```

```

In [23]: df.describe()

```

Out[23]:

	transaction_id	year	transaction_qty	store_id	product_id	unit
count	149116.000000	149116.0	149116.000000	149116.000000	149116.000000	149116.0
mean	74737.371872	2025.0	1.438276	5.342063	47.918607	3.3
std	43153.600016	0.0	0.542509	2.074241	17.930020	2.6
min	1.000000	2025.0	1.000000	3.000000	1.000000	0.8
25%	37335.750000	2025.0	1.000000	3.000000	33.000000	2.5
50%	74727.500000	2025.0	1.000000	5.000000	47.000000	3.0
75%	112094.250000	2025.0	2.000000	8.000000	60.000000	3.7
max	149456.000000	2025.0	8.000000	8.000000	87.000000	45.0

In [25]:

```
df["Revenue"] = df["transaction_qty"] * df["unit_price"]
```

In [26]:

```
df.head()
```

Out[26]:

	transaction_id	year	transaction_time	transaction_qty	store_id	store_location	prod
0	1	2025	07:06:11	2	5	Lower Manhattan	
1	2	2025	07:08:56	2	5	Lower Manhattan	
2	3	2025	07:14:04	2	5	Lower Manhattan	
3	4	2025	07:20:24	1	5	Lower Manhattan	
4	5	2025	07:22:41	2	5	Lower Manhattan	

In [27]:

```
total_revenue = df["Revenue"].sum()

print(f"Total Revenue : ${total_revenue:,.2f}")
```

Total Revenue : \$698,812.33

In [28]:

```
total_qty = df["transaction_qty"].sum()

print(f"Total Units Sold : {total_qty:,}")
```

Total Units Sold : 214,470

In [29]:

```
print("Unique Products :", df["product_detail"].nunique())
```

Unique Products : 80

```
In [30]: print("Categories :", df["product_category"].nunique())
```

Categories : 9

```
In [31]: print(df["store_location"].unique())
```

['Lower Manhattan' 'Hell's Kitchen' 'Astoria']

```
In [32]: import matplotlib.pyplot as plt
```

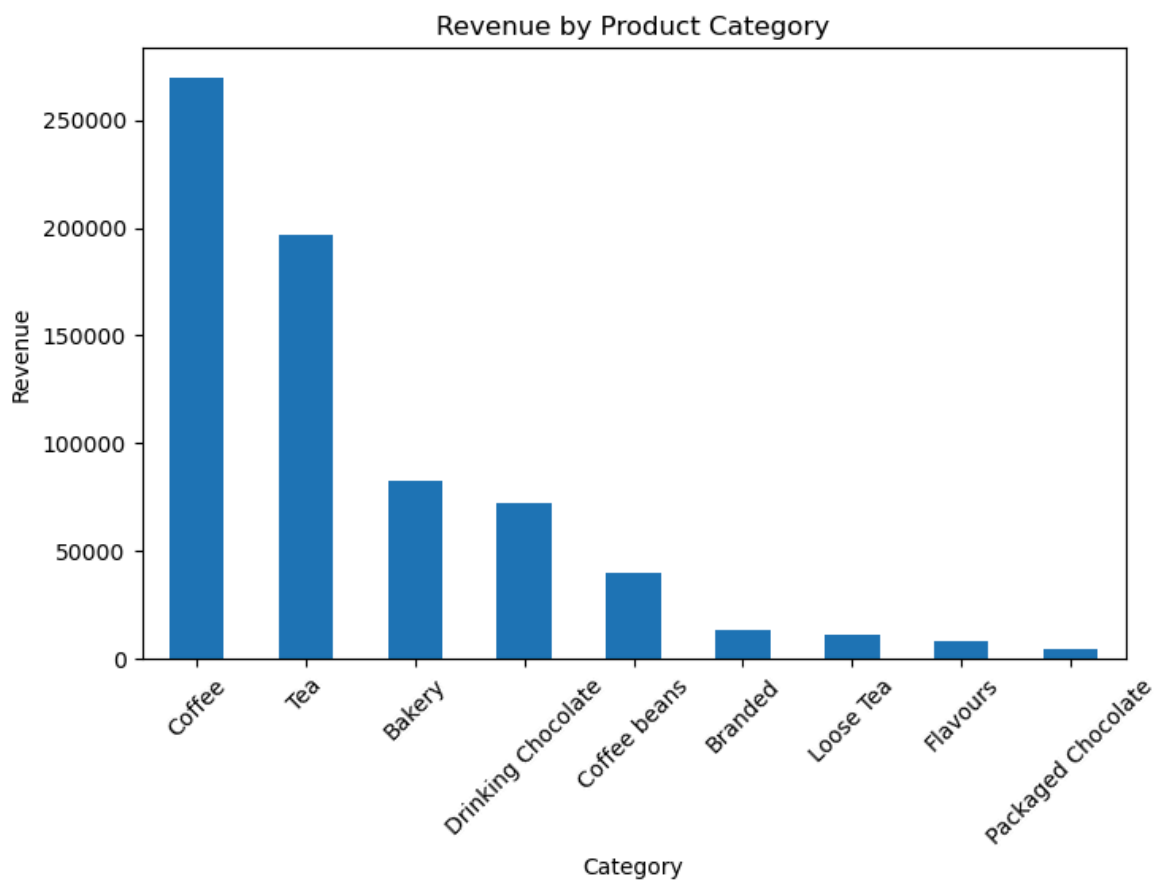
```
category_sales = (  
    df.groupby("product_category")["Revenue"]  
      .sum()  
      .sort_values(ascending=False)  
)
```

```
plt.figure(figsize=(8,5))  
category_sales.plot(kind="bar")
```

```
plt.title("Revenue by Product Category")  
plt.xlabel("Category")  
plt.ylabel("Revenue")
```

```
plt.xticks(rotation=45)
```

```
plt.show()
```



Business Insight

- Coffee contributes the highest revenue among all product categories.
- Tea and Bakery contribute comparatively less.

- The company should continue investing in high-performing categories while improving the performance of lower-revenue categories.

Product Popularity Analysis

This analysis identifies the best-selling and least-selling products based on transaction quantity.

```
In [33]: product_sales = (  
    df.groupby("product_detail")["transaction_qty"]  
        .sum()  
        .sort_values(ascending=False)  
    )  
  
product_sales.head(10)
```

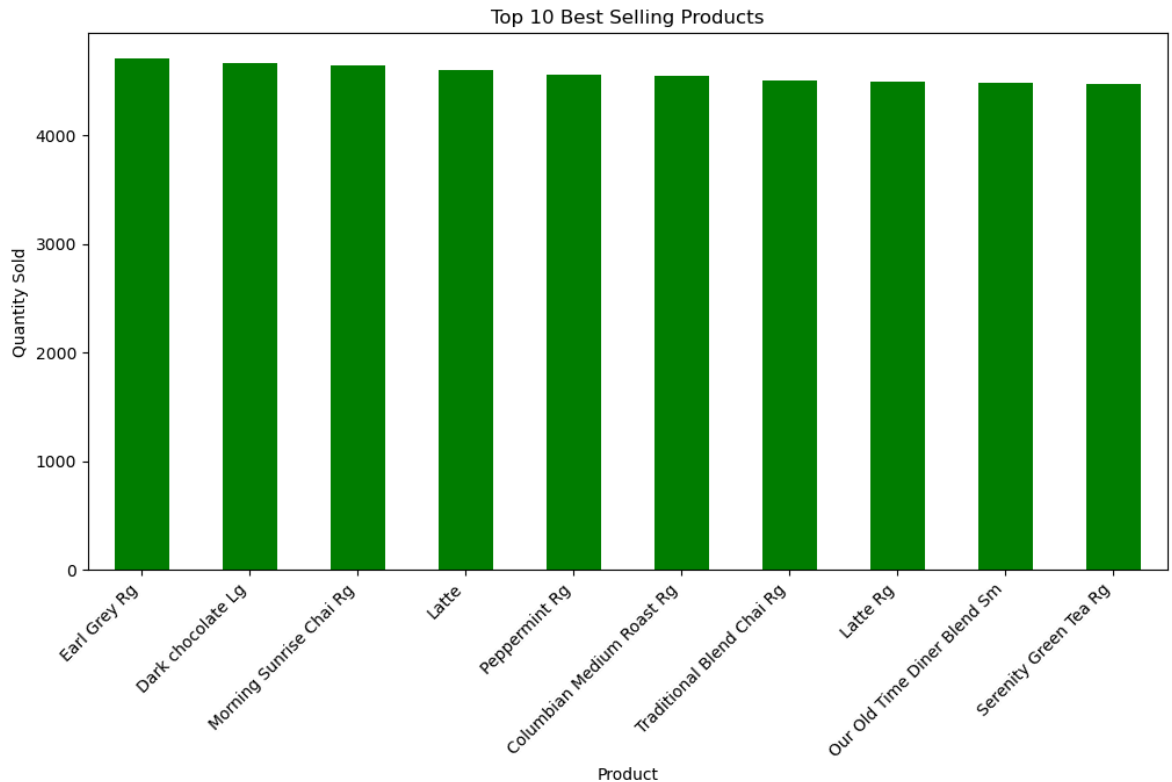
```
Out[33]: product_detail  
Earl Grey Rg          4708  
Dark chocolate Lg     4668  
Morning Sunrise Chai Rg 4643  
Latte                 4602  
Peppermint Rg         4564  
Columbian Medium Roast Rg 4547  
Traditional Blend Chai Rg 4512  
Latte Rg              4497  
Our Old Time Diner Blend Sm 4484  
Serenity Green Tea Rg    4477  
Name: transaction_qty, dtype: int64
```

```
In [34]: top10 = product_sales.head(10)  
  
top10
```

```
Out[34]: product_detail  
Earl Grey Rg          4708  
Dark chocolate Lg     4668  
Morning Sunrise Chai Rg 4643  
Latte                 4602  
Peppermint Rg         4564  
Columbian Medium Roast Rg 4547  
Traditional Blend Chai Rg 4512  
Latte Rg              4497  
Our Old Time Diner Blend Sm 4484  
Serenity Green Tea Rg    4477  
Name: transaction_qty, dtype: int64
```

```
In [35]: plt.figure(figsize=(12,6))  
  
top10.plot(kind="bar", color="green")  
  
plt.title("Top 10 Best Selling Products")  
plt.xlabel("Product")  
plt.ylabel("Quantity Sold")  
  
plt.xticks(rotation=45, ha="right")
```

```
plt.show()
```



Business Insight

The above chart identifies the ten most popular products based on sales quantity.

These products represent customer favourites and should always remain available in stock. They can also be highlighted in promotions and marketing campaigns to maximize sales.

```
In [36]: bottom10 = product_sales.tail(10)

bottom10
```

```
Out[36]: product_detail
Traditional Blend Chai      153
Lemon Grass                 152
Primo Espresso Roast       150
Columbian Medium Roast     148
Chili Mayan                 148
Jamacian Coffee River      146
Earl Grey                   142
Guatemalan Sustainably Grown 134
Spicy Eye Opener Chai      122
Dark chocolate              118
Name: transaction_qty, dtype: int64
```

```
In [37]: plt.figure(figsize=(12,6))

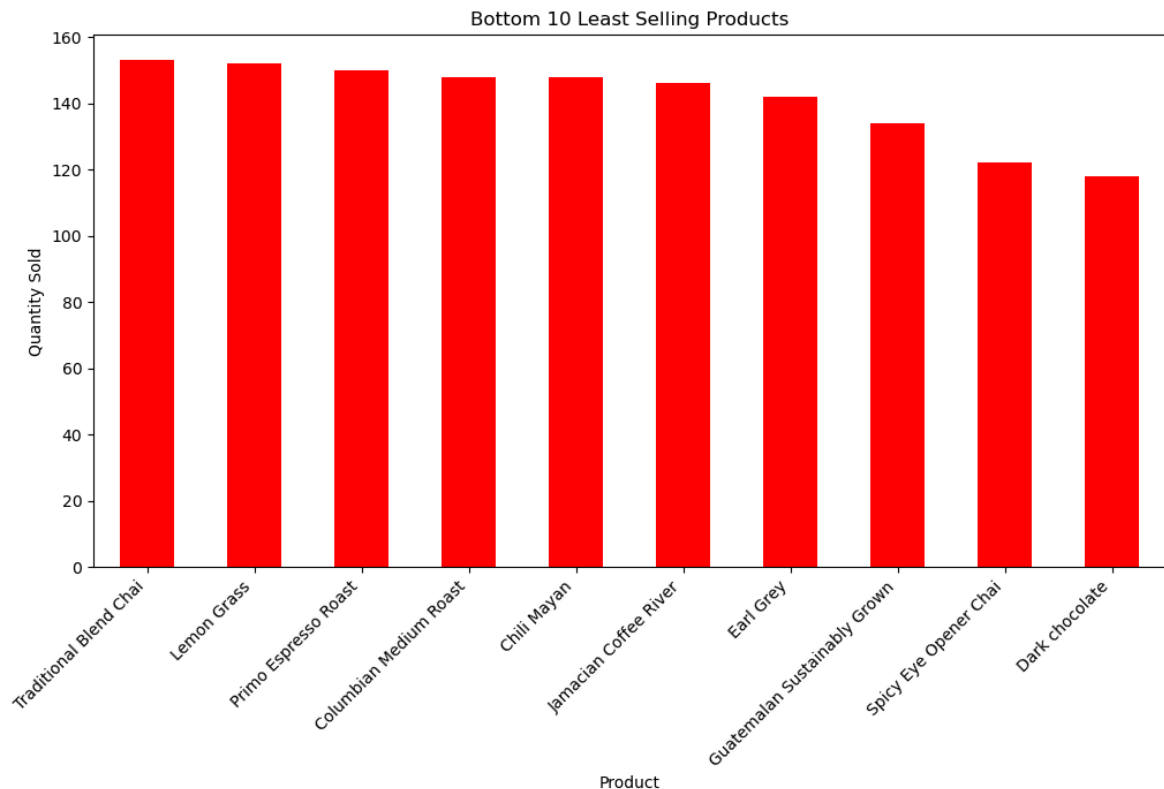
bottom10.plot(kind="bar", color="red")

plt.title("Bottom 10 Least Selling Products")
```

```
plt.xlabel("Product")
plt.ylabel("Quantity Sold")

plt.xticks(rotation=45, ha="right")

plt.show()
```



Business Insight

These products have the lowest sales volume.

Management should investigate whether these products need better marketing, pricing adjustments, recipe improvements, or removal from the menu.

Revenue Contribution Analysis

This analysis identifies which products contribute the most revenue and compares product popularity with revenue generation.

```
In [39]: product_revenue = (
    df.groupby("product_detail")["Revenue"]
      .sum()
      .sort_values(ascending=False)
    )

product_revenue.head(10)
```



```
Out[39]: product_detail
Sustainably Grown Organic Lg      21151.75
Dark chocolate Lg                  21006.00
Latte Rg                           19112.25
Cappuccino Lg                     17641.75
Morning Sunrise Chai Lg           17384.00
Latte                              17257.50
Jamaican Coffee River Lg          16481.25
Sustainably Grown Organic Rg      16233.75
Cappuccino                        15997.50
Brazilian Lg                      15109.50
Name: Revenue, dtype: float64
```

```
In [40]: top10_revenue = product_revenue.head(10)

top10_revenue
```

```
Out[40]: product_detail
Sustainably Grown Organic Lg      21151.75
Dark chocolate Lg                  21006.00
Latte Rg                           19112.25
Cappuccino Lg                     17641.75
Morning Sunrise Chai Lg           17384.00
Latte                              17257.50
Jamaican Coffee River Lg          16481.25
Sustainably Grown Organic Rg      16233.75
Cappuccino                        15997.50
Brazilian Lg                      15109.50
Name: Revenue, dtype: float64
```

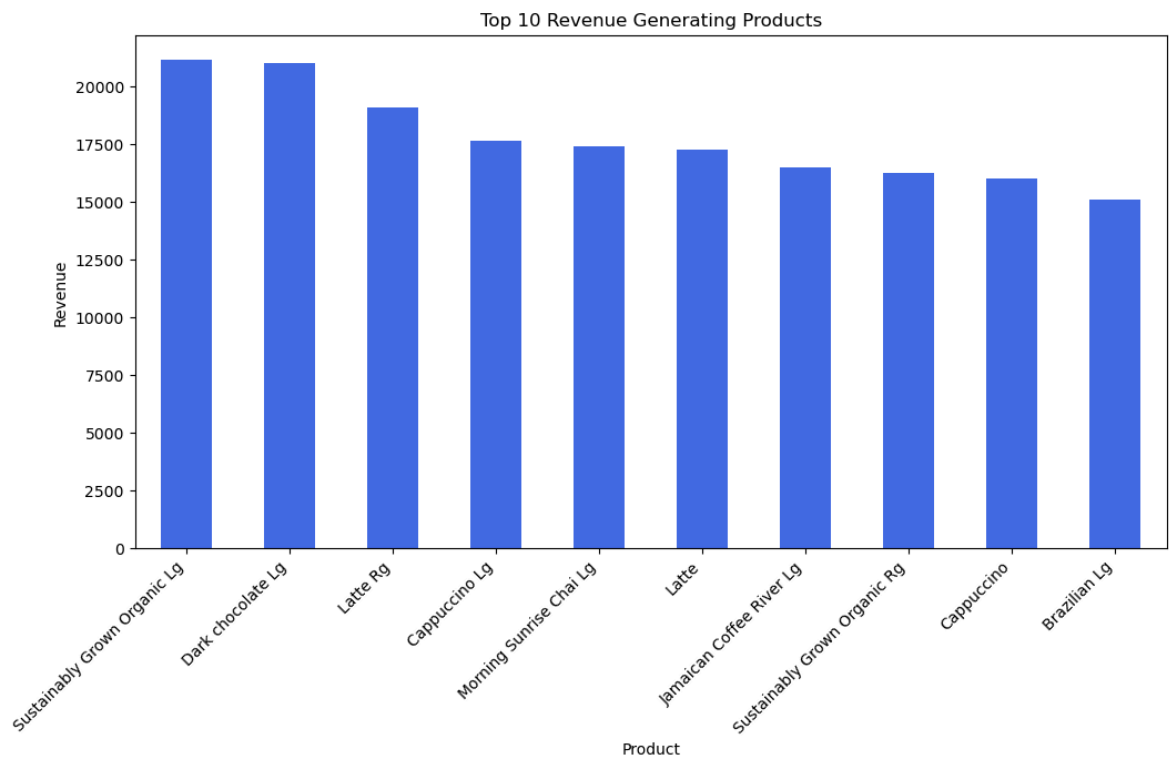
```
In [41]: plt.figure(figsize=(12,6))

top10_revenue.plot(kind="bar", color="royalblue")

plt.title("Top 10 Revenue Generating Products")
plt.xlabel("Product")
plt.ylabel("Revenue")

plt.xticks(rotation=45, ha="right")

plt.show()
```



Business Insight

- These products generate the highest revenue for the company.
- They are the primary contributors to business profitability.
- The company should ensure continuous availability of these products and prioritize them in promotions and inventory planning.

```
In [43]: revenue_share = (
          df.groupby("product_detail")["Revenue"]
            .sum()
            .sort_values(ascending=False)
          )

revenue_share = (
    revenue_share / revenue_share.sum() * 100
).round(2)

revenue_share.head(10)
```

```
Out[43]: product_detail
Sustainably Grown Organic Lg    3.03
Dark chocolate Lg              3.01
Latte Rg                      2.73
Cappuccino Lg                 2.52
Morning Sunrise Chai Lg       2.49
Latte                         2.47
Jamaican Coffee River Lg      2.36
Sustainably Grown Organic Rg  2.32
Cappuccino                    2.29
Brazilian Lg                   2.16
Name: Revenue, dtype: float64
```

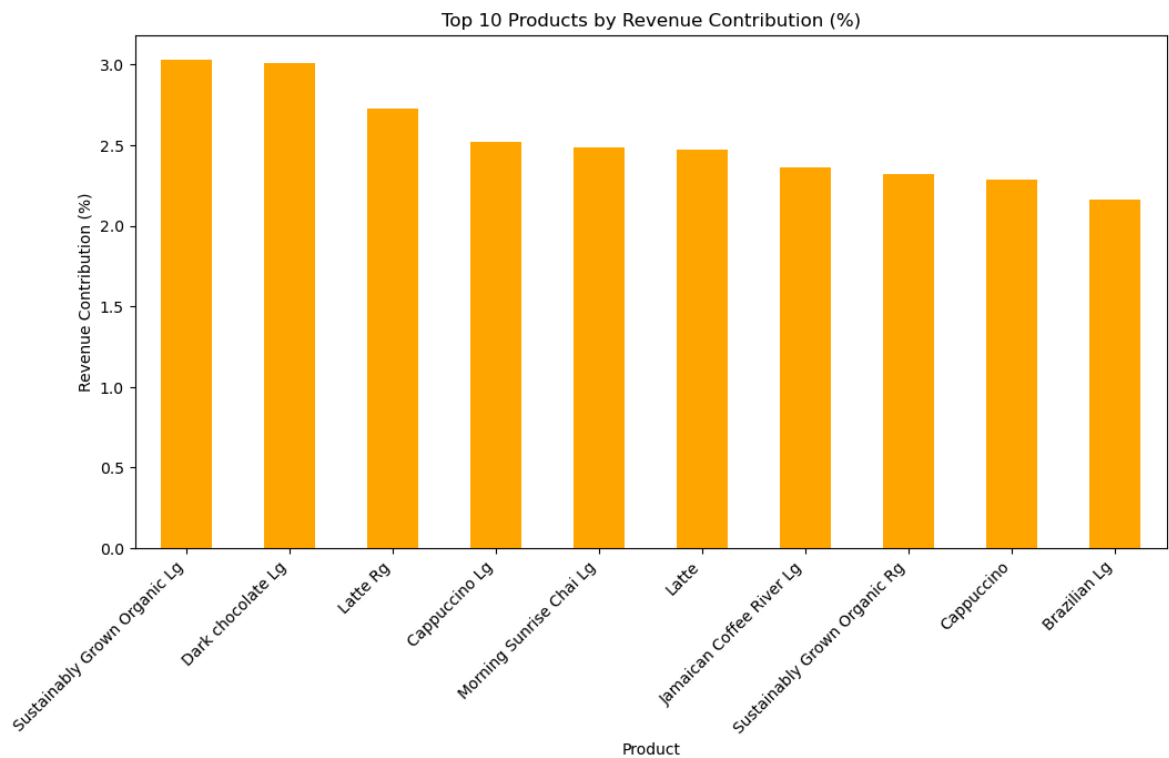
```
In [44]: plt.figure(figsize=(12,6))
```

```
revenue_share.head(10).plot(kind="bar", color="orange")

plt.title("Top 10 Products by Revenue Contribution (%)")
plt.xlabel("Product")
plt.ylabel("Revenue Contribution (%)")

plt.xticks(rotation=45, ha="right")

plt.show()
```



Business Insight

- A small number of products contribute a significant share of total revenue.
- These products are strategic assets for the company.
- Increasing their availability and customer visibility can further improve revenue.

```
In [46]: comparison = df.groupby("product_detail").agg(
    Quantity=("transaction_qty", "sum"),
    Revenue=("Revenue", "sum")
)

comparison.head()
```

Out[46]:

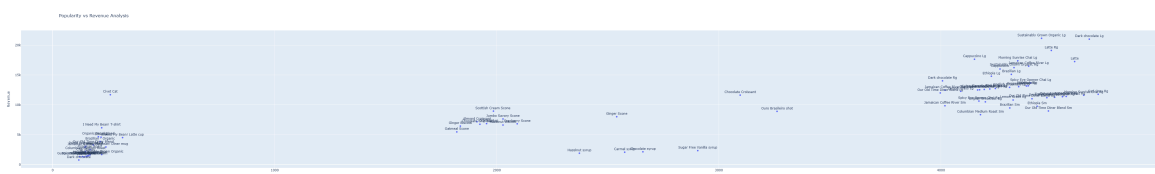
	Quantity	Revenue
product_detail		
Almond Croissant	1911	7168.13
Brazilian - Organic	214	3852.00
Brazilian Lg	4317	15109.50
Brazilian Rg	4385	13155.00
Brazilian Sm	4310	9482.00

In [47]: `import plotly.express as px`

```
fig = px.scatter(
    comparison,
    x="Quantity",
    y="Revenue",
    text=comparison.index,
    title="Popularity vs Revenue Analysis",
    height=700
)

fig.update_traces(textposition="top center")

fig.show()
```



Business Insight

- Products in the top-right corner are both popular and highly profitable.
- Products with high sales but low revenue may require pricing review.
- Products with high revenue but low sales may benefit from marketing campaigns.
- Products in the bottom-left corner should be evaluated for menu optimization.

Category & Product-Type Performance

This section evaluates revenue contribution by product category and product type to identify the strongest-performing business segments.

```
In [50]: product_type_revenue = (
    df.groupby("product_type")["Revenue"]
        .sum()
        .sort_values(ascending=False)
    )

product_type_revenue.head(15)
```

```
Out[50]: product_type
Barista Espresso      91406.20
Brewed Chai tea       77081.95
Hot chocolate         72416.00
Gourmet brewed coffee 70034.60
Brewed Black tea      47932.00
Brewed herbal tea     47539.50
Premium brewed coffee 38781.15
Organic brewed coffee 37746.50
Scone                 36866.12
Drip coffee           31984.00
Pastry                25655.99
Brewed Green tea      23852.50
Biscotti              19793.53
Premium Beans         14583.50
Organic Beans          8509.50
Name: Revenue, dtype: float64
```

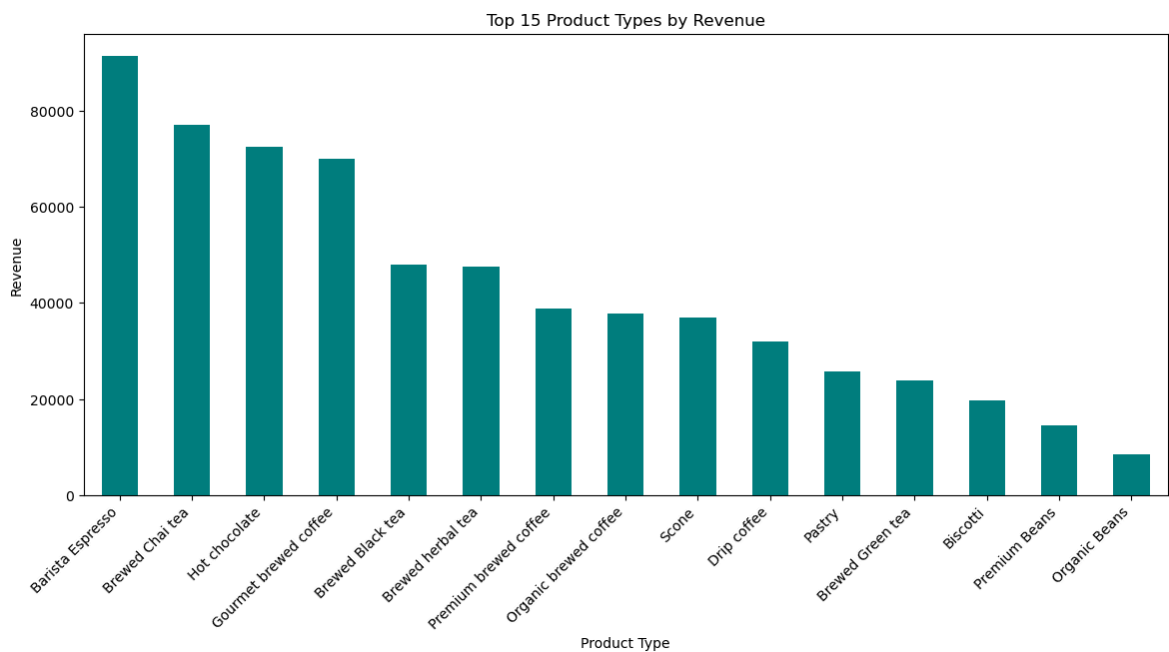
```
In [51]: plt.figure(figsize=(14,6))

product_type_revenue.head(15).plot(
    kind="bar",
    color="teal"
)

plt.title("Top 15 Product Types by Revenue")
plt.xlabel("Product Type")
plt.ylabel("Revenue")

plt.xticks(rotation=45, ha="right")

plt.show()
```



Business Insight

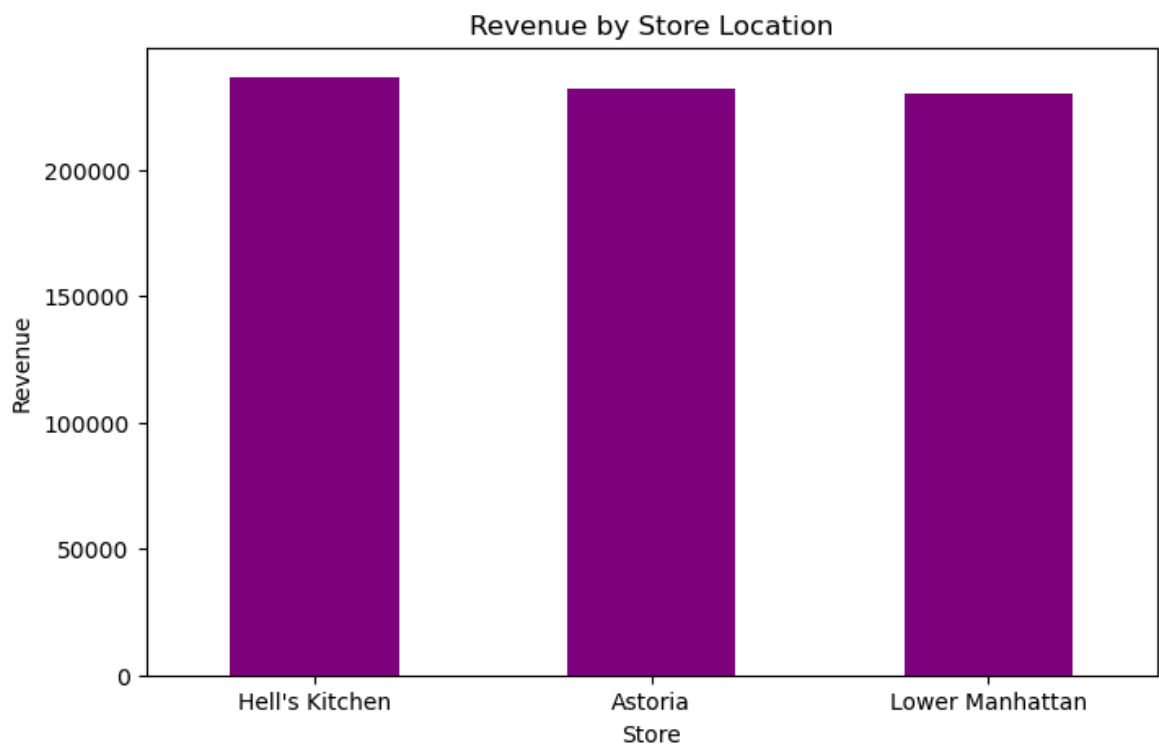
- The chart identifies the product types generating the highest revenue.

- These product types should receive priority in inventory management and promotions.
- Lower-performing product types can be reviewed for pricing or marketing improvements.

```
In [53]: store_revenue = (  
    df.groupby("store_location")["Revenue"]  
        .sum()  
        .sort_values(ascending=False)  
    )  
  
store_revenue
```

```
Out[53]: store_location  
Hell's Kitchen    236511.17  
Astoria          232243.91  
Lower Manhattan  230057.25  
Name: Revenue, dtype: float64
```

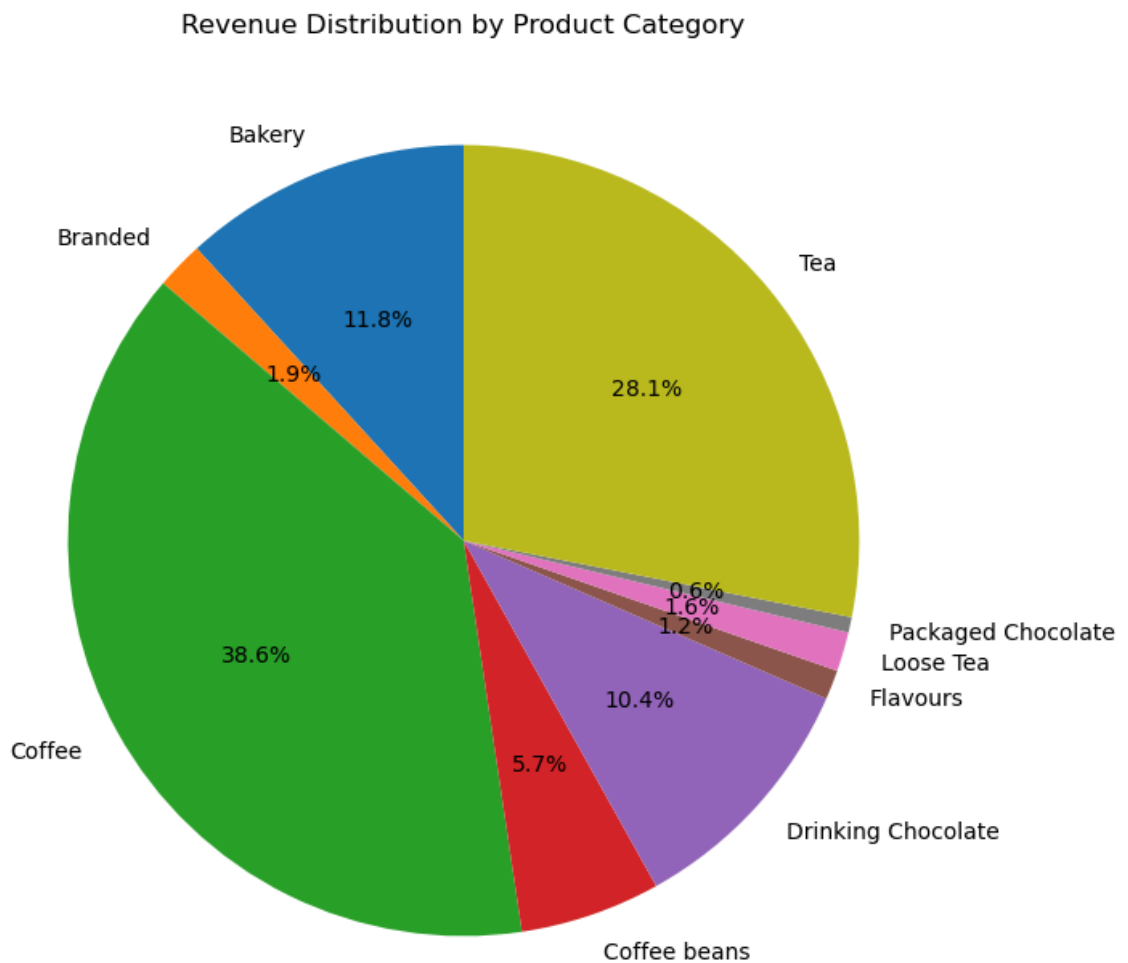
```
In [54]: plt.figure(figsize=(8,5))  
  
store_revenue.plot(  
    kind="bar",  
    color="purple"  
)  
  
plt.title("Revenue by Store Location")  
plt.xlabel("Store")  
plt.ylabel("Revenue")  
  
plt.xticks(rotation=0)  
  
plt.show()
```



Business Insight

- This analysis compares the revenue generated by each store location.
- The highest-performing store can serve as a benchmark for business strategies.
- Lower-performing stores should be analyzed for customer demand and operational improvements.

```
In [55]: category_revenue = (  
    df.groupby("product_category")["Revenue"]  
        .sum()  
    )  
  
plt.figure(figsize=(8,8))  
  
plt.pie(  
    category_revenue,  
    labels=category_revenue.index,  
    autopct="%1.1f%%",  
    startangle=90  
    )  
  
plt.title("Revenue Distribution by Product Category")  
  
plt.show()
```



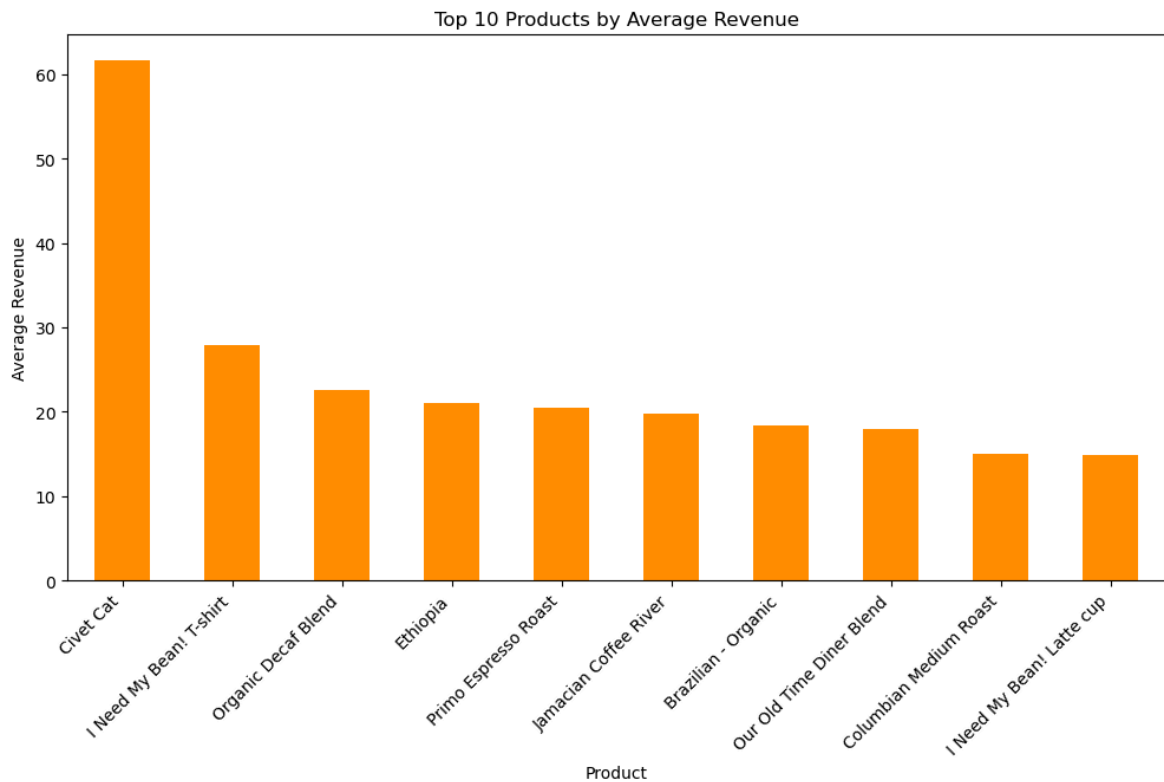
Business Insight

- The pie chart illustrates the percentage contribution of each category to total revenue.
- Categories with larger shares are the primary revenue drivers.
- Smaller categories may require promotional campaigns or product diversification.

```
In [56]: avg_revenue = (  
    df.groupby("product_detail")["Revenue"]  
        .mean()  
        .sort_values(ascending=False)  
    )  
  
avg_revenue.head(10)
```

```
Out[56]: product_detail  
Civet Cat                61.578947  
I Need My Bean! T-shirt  27.886878  
Organic Decaf Blend      22.609223  
Ethiopia                 21.000000  
Primo Espresso Roast    20.450000  
Jamacian Coffee River   19.750000  
Brazilian - Organic      18.430622  
Our Old Time Diner Blend 18.000000  
Columbian Medium Roast  15.000000  
I Need My Bean! Latte cup 14.832237  
Name: Revenue, dtype: float64
```

```
In [57]: plt.figure(figsize=(12,6))  
  
avg_revenue.head(10).plot(  
    kind="bar",  
    color="darkorange"  
)  
  
plt.title("Top 10 Products by Average Revenue")  
plt.xlabel("Product")  
plt.ylabel("Average Revenue")  
  
plt.xticks(rotation=45, ha="right")  
  
plt.show()
```

Business Insight

- These products generate the highest average revenue per transaction.
- They can be considered premium products and should be highlighted through marketing strategies.

Revenue Concentration (Pareto 80/20 Analysis)

```
In [58]: pareto = (
    df.groupby("product_detail")["Revenue"]
      .sum()
      .sort_values(ascending=False)
    )

pareto_df = pareto.reset_index()
pareto_df.columns = ["Product", "Revenue"]

pareto_df["Revenue %"] = (
    pareto_df["Revenue"] / pareto_df["Revenue"].sum() * 100
)

pareto_df["Cumulative %"] = pareto_df["Revenue %"].cumsum()

pareto_df.head()
```

Out[58]:

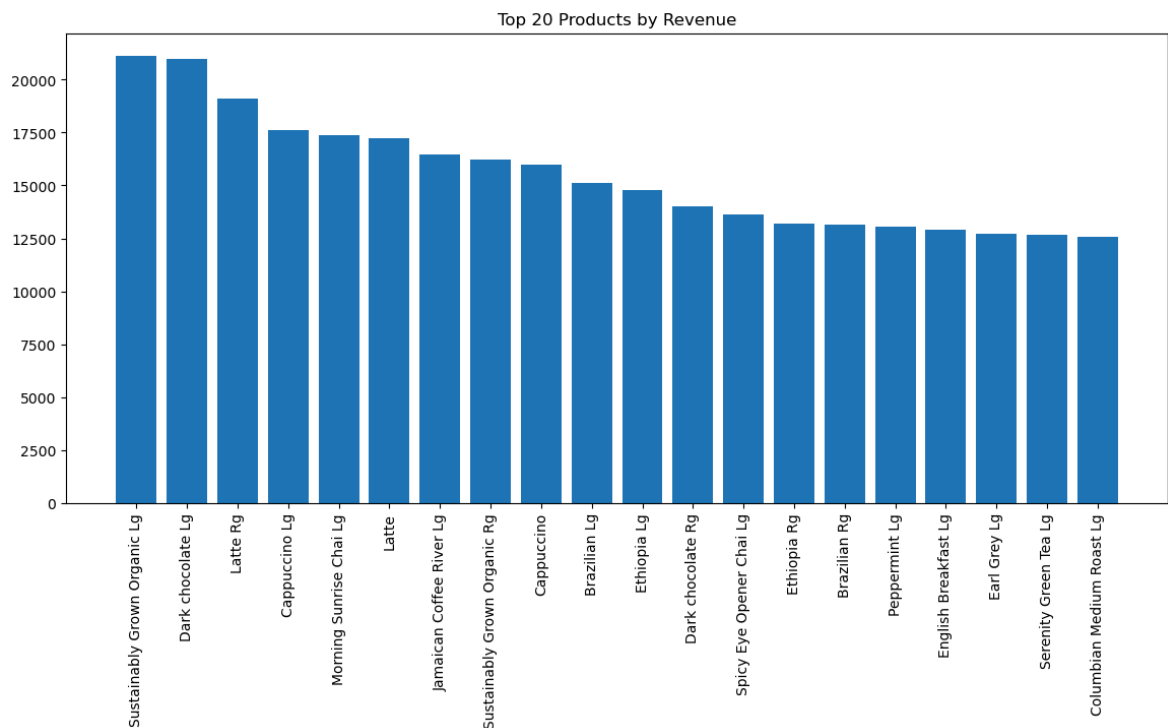
	Product	Revenue	Revenue %	Cumulative %
0	Sustainably Grown Organic Lg	21151.75	3.026814	3.026814
1	Dark chocolate Lg	21006.00	3.005957	6.032771
2	Latte Rg	19112.25	2.734962	8.767733
3	Cappuccino Lg	17641.75	2.524533	11.292266
4	Morning Sunrise Chai Lg	17384.00	2.487649	13.779916

```
In [59]: plt.figure(figsize=(14,6))

plt.bar(
    pareto_df["Product"][:20],
    pareto_df["Revenue"][:20]
)

plt.xticks(rotation=90)
plt.title("Top 20 Products by Revenue")

plt.show()
```



```
In [60]: plt.figure(figsize=(14,6))

plt.plot(
    pareto_df["Cumulative %"][:20],
    marker="o",
    color="red"
)

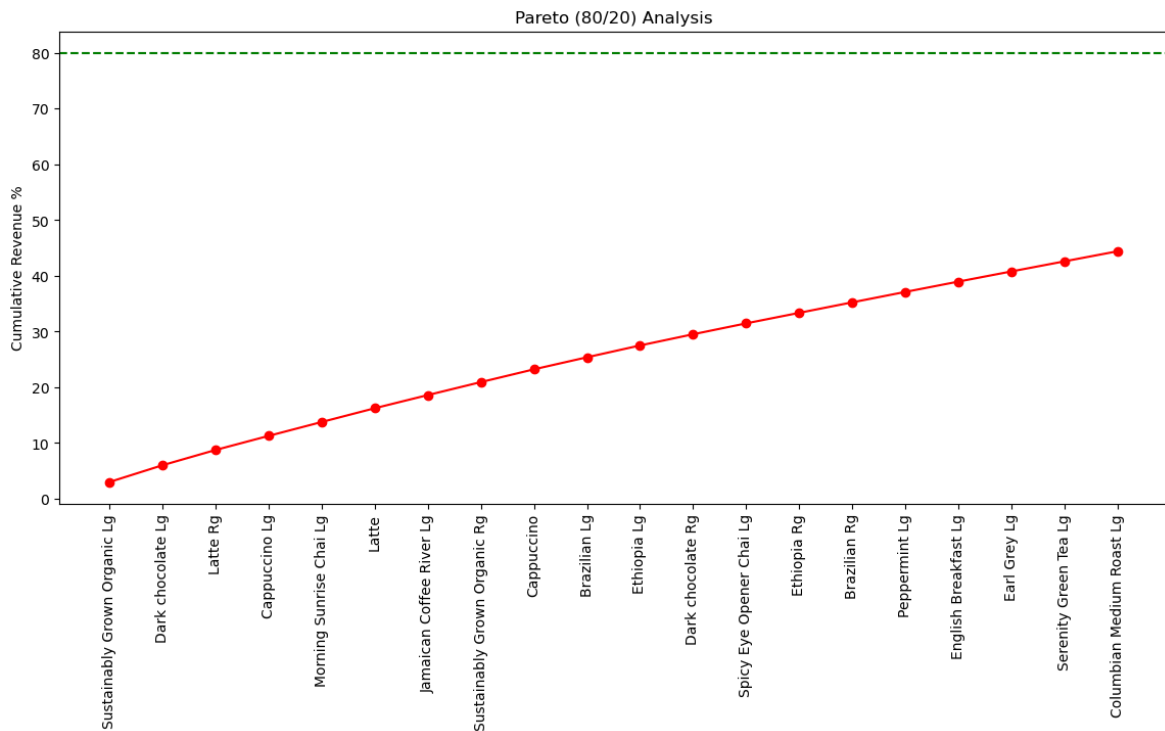
plt.axhline(80, color="green", linestyle="--")

plt.xticks(range(20), pareto_df["Product"][:20], rotation=90)

plt.title("Pareto (80/20) Analysis")
```

```
plt.ylabel("Cumulative Revenue %")

plt.show()
```



```
In [61]: print("="*40)

print("TOTAL REVENUE")
print(df["Revenue"].sum())

print()

print("TOTAL QUANTITY SOLD")
print(df["transaction_qty"].sum())

print()

print("TOTAL PRODUCTS")
print(df["product_detail"].nunique())

print()

print("TOTAL CATEGORIES")
print(df["product_category"].nunique())

print()

print("TOTAL STORES")
print(df["store_location"].nunique())

print("="*40)
```

```
=====
TOTAL REVENUE
698812.3300000001

TOTAL QUANTITY SOLD
214470

TOTAL PRODUCTS
80

TOTAL CATEGORIES
9

TOTAL STORES
3
=====
```